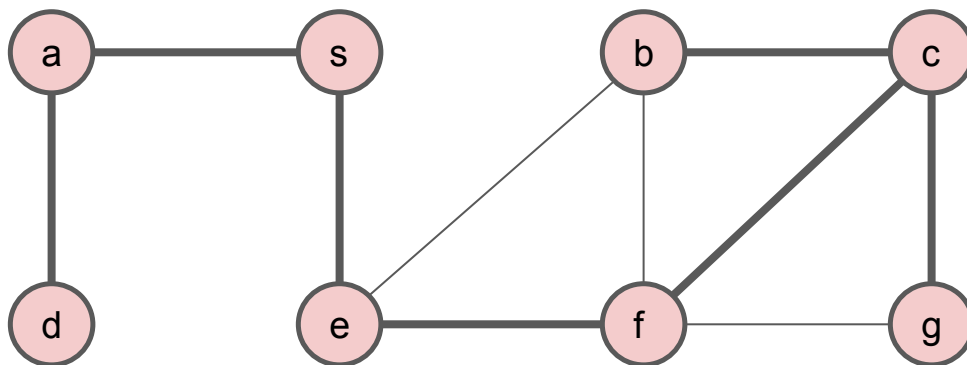


Busca em profundidade

Busca em profundidade



Pilha:
s
vazia
s e
s
s e f
s e
s e f c
s e f
s e f c g
s e f c
s e f
s e f c b
s e f c
s e f
s e
s
vazia
s a
s
s a d
s a
s
vazia

Busca em profundidade

- Busca genérica com pilha
 - Nós alcançáveis a partir de s são encontrados, independente da estrutura de dados
 - Empilhamos o nó encontrado v e seu predecessor u
 - Pois u foi desempilhado quando a execução voltou para o nó u
 - Para retomar o acesso aos vizinhos do ponto que parou
 - Empilhamos o nó u e a quantidade i de arestas saindo de u que já foram acessadas
 - Ex.: $(u,3)$ indica que os 3 primeiros vizinhos de u já foram acessados na tarefa de percorrer os vizinhos de u

Busca em profundidade

- Invariante do loop
 - IL1: Os nós na pilha formam um caminho iniciando em s
 - IL2: Cada tupla (u,i) na pilha indica que as i primeiras arestas saindo de u já foram acessadas
Um nó não aparece em mais de uma tupla na pilha.
 - Pilha contém todos os encontrados não finalizados
 - Não encontrados não ocorrem na pilha
 - Nós finalizados são os encontrados que não estão na pilha
 - Entra na pilha assim que é encontrado
 - Finalizados saem da pilha, e não retornam

Caminho mais curto com busca em largura

algoritmo BuscaProfundidade(G, s)

<pre-cond>: G é um grafo (direcionado ou não), e s é um de seus nós

<pos-cond>: P/ cada nó v , $\pi(v)$ é o pai de v na árvore de busca em profundidade.

 encontrado(v) == True sse v é alcançável a partir de s .

para cada nó v em G , faça encontrado(v) = False

P = pilha vazia

$P.push((s, 0))$; encontrado(s) = True

loop

 <inv-loop>: IL1 e IL2.

 saia quando P é vazia

$(u, i) = P.pop()$

 se u tem um $(i+1)$ -ésimo vizinho v

$P.push((u, i+1))$

 se encontrado(v) == False

 encontrado(v) = True; $\pi(v) = u$

$P.push((v, 0))$

Busca em profundidade

- Mantendo o invariante do loop
 - IL1: Os nós na pilha formam um caminho iniciando em s
 - IL2: Cada tupla (u,i) na pilha indica que as i primeiras arestas saindo de u já foram acessadas
Um nó não aparece em mais de uma tupla na pilha.
 - Tupla (u,i) é desempilhada
 - Se u tem o $(i+1)$ -ésimo vizinho v
 - Empilhamos $(u,i+1)$.
 - Por IL2', já acessamos as i primeiras arestas de u
 - E nesta iteração acessamos a aresta $i+1$
 - Então satisfaz IL2''. Também IL1'', por IL1' e por u estar inicialmente na pilha.
 - Se v é “não encontrado”, empilhamos $(v,0)$
 - Como existe aresta (u,v) , u precede v na fila, e existe caminho de s até u (IL1'),
 - Então nós da pilha formam caminho de s até v (satisfaz IL1'')
 - Como v é “não encontrado”, zero arestas de v foram acessadas (satisfaz IL2'')

Busca em profundidade

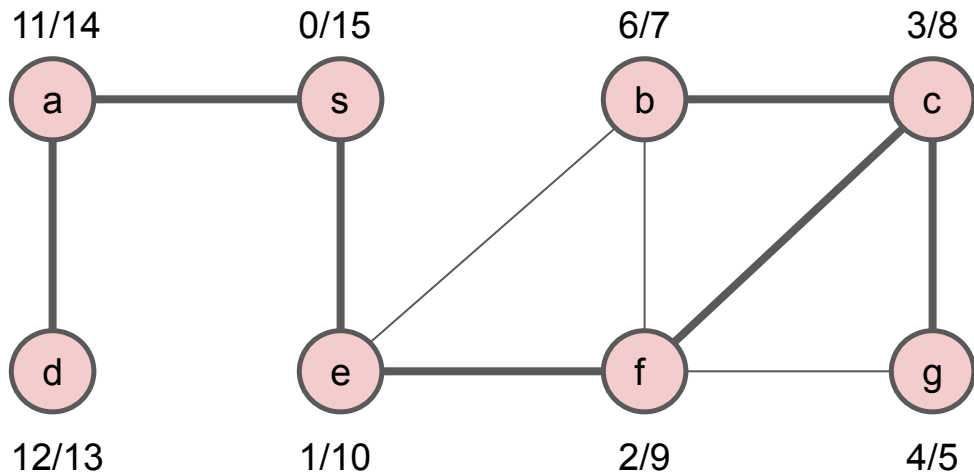
- Estabelecendo o invariante do loop
 - IL1: Os nós na pilha formam um caminho iniciando em s
 - IL2: Cada tupla (u,i) na pilha indica que as i primeiras arestas saindo de u já foram acessadas
Um nó não aparece em mais de uma tupla na pilha.
 - Apenas $(s,0)$ está na pilha
 - IL1: satisfeito (s é o único nó no caminho)
 - IL2: satisfeito (nenhum vizinho de s foi acessado)

Busca em profundidade

- Estabelecendo as pós-condições
 - IL1: Os nós na pilha formam um caminho iniciando em s
 - IL2: Cada tupla (u,i) na pilha indica que as i primeiras arestas saindo de u já foram acessadas
Um nó não aparece em mais de uma tupla na pilha.
- Por IL1, todo nó colocado na pilha é alcançável a partir de s
- Resta mostrar que todo nó alcançável a partir de s passa pela pilha
 - Para ter um nó z alcançável mas não encontrado
 - Temos que ter um caminho de s até z tal que algum nó encontrado não teve todos seus vizinhos encontrados
 - Basta mostrar que todo nó encontrado teve todos seus vizinhos encontrados
 - Tupla retirada (u,i) não gera retorno de u apenas quando $i > \text{núm. de vizinhos de } u$
 - Então pela condição de saída e IL2
 - Todo nó que passou pela pilha teve todos os vizinhos encontrados

Busca em profundidade - marcação de tempo

- Um relógio “tempo” é inicializado com zero e incrementado
 - Quando um nó é encontrado
 - Quando não é re-empilhado (finalizado)
- Podemos então guardar em
 - $s(v)$ o instante que v foi encontrado
 - $f(v)$ o instante que v foi finalizado



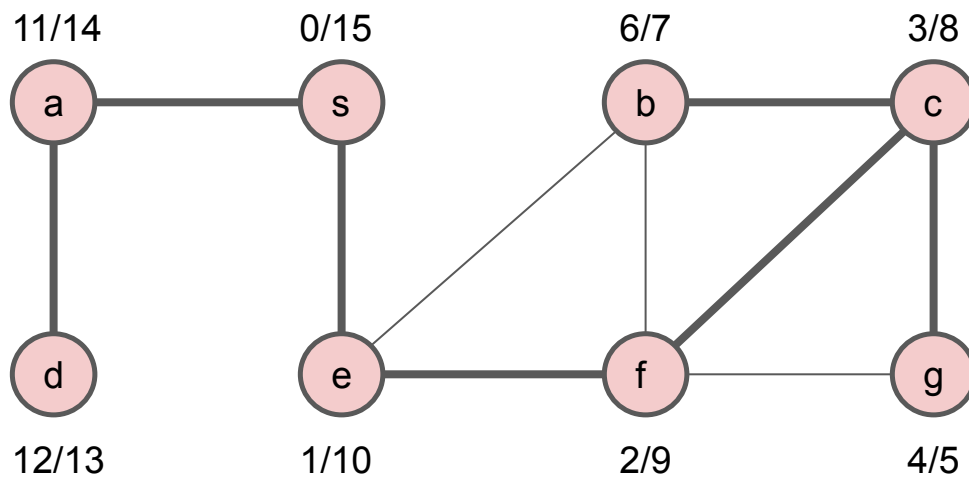
Busca em profundidade - marcação de tempo

```
algoritmo BuscaProfundidade(G,s)
  para cada nó v em G, faça encontrado(v) == False
  P = pilha vazia; tempo = 0
  P.push((s,0)); encontrado(s) = True
  loop
    saia quando P é vazia
    (u,i) = P.pop()
    se u tem um (i+1)-ésimo vizinho v
      P.push((u,i+1))
      se encontrado(v) == False
        s(v) = tempo; tempo = tempo + 1
        encontrado(v) = True;  $\pi(v)$  = u
        P.push((v,0))
  senão
    f(u) = tempo; tempo = tempo + 1
```

Busca em profundidade - marcação de tempo

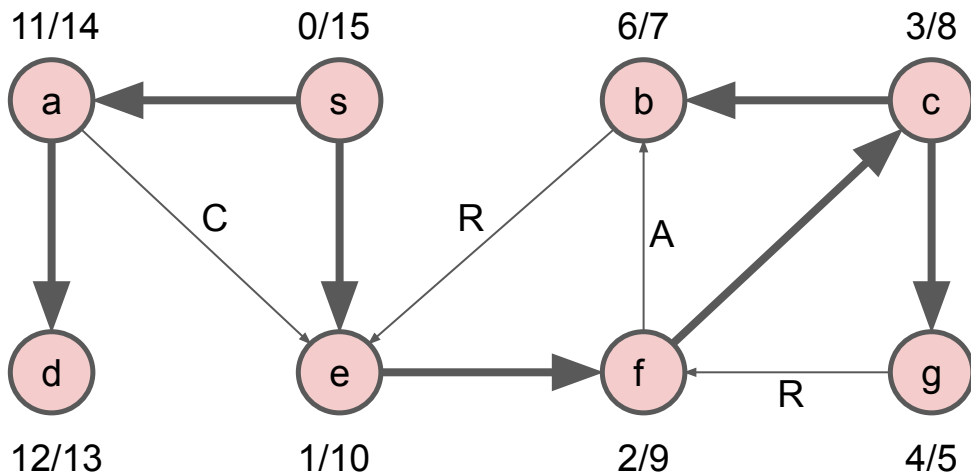
- Permite determinar descendência em tempo constante
 - v é descendente de u na ABP sse o intervalo $[s(v), f(v)]$ está contido no intervalo $[s(u), f(u)]$.
Ou seja, $s(u) < s(v) < f(v) < f(u)$.
 - $s(v)$: instante v entra pela 1ª vez. $f(v)$: instante v sai pela última vez.
 - Quando v sai e volta para pilha, os ancestrais/s/f permanecem inalterados (ignorar)
 - Nós na pilha estão em ordem crescente de s (relógio incrementado)
 - Nós finalizam (deixam a pilha) em ordem crescente de f (relógio incrementado)
 - Por IL1, os nós na pilha são os ancestrais de v na ABP
 - Quando u não é ancestral de v , u terminou antes de v começar, ou ainda não começou
 - Ou seja, os intervalos são disjuntos (sem interseção)

Busca em profundidade - marcação de tempo



Busca em profundidade - classificação de aresta (u,v)

- Árvore (negrito): determinadas por π (**v é “não encontrado”**)
- Avanço (A): para um descendente (**s(u) < s(v) < f(v) < f(u)**)
- Retorno (R): para um ancestral (**s(v) < s(u) < f(u) < f(v)**) (**f(v) não foi atribuído**)
- Cruzada (C): nenhuma das anteriores (para finalizado: **s(v) < f(v) < s(u) < f(u)**) (**f(v) foi atribuído**)



Grafos não direcionados só possuem de árvore e de retorno.

Busca em profundidade - marcação de tempo

```
algoritmo BuscaProfundidade(G,s)
  para cada nó v em G, faça encontrado(v) == False
  P = pilha vazia; tempo = 0; P.push((s,0)); encontrado(s) = True
  loop
    saia quando S é vazia
    (u,i) = P.pop()
    se u tem um (i+1)-ésimo vizinho v
      P.push((u,i+1))
      se encontrado(v) == False
        s(v) = tempo; tempo = tempo + 1; tipo(u,v) = "árvore"
        encontrado(v) = True; π(v) = u; P.push((v,0))
      senão, se s(u) < s(v), faça tipo(u,v) = "avanço"
      senão, se f(v) não foi atribuído, faça tipo(u,v) = "retorno"
      senão, faça tipo(u,v) = "cruzada"
  senão
    f(v) = tempo; tempo = tempo + 1
```

Busca em profundidade - recursivo

- A versão recursiva é mais simples, aproveita a pilha de stack frames
 - Um stack frame para cada nó u
 - Retoma do ponto que parou na tarefa de percorrer os vizinhos de u
- “Caminho de não encontrados”: formado apenas por nós não encontrados

Busca em profundidade - recursivo

algoritmo **BuscaProfundidadeRecursivo**(G,u)

<pre-cond>: G é um grafo (direcionado ou não), e u é um de seus nós.

encontrado(v) é um campo booleano de cada nó v.

<pos-cond>: todo nó v alcançável a partir de u por "caminho de não encontrados"
terá encontrado(v) == True.

se encontrado(u) == True

 não faz nada

senão

 encontrado(u) = True

 para cada vizinho v de u em G

BuscaProfundidadeRecursivo(G,v)

algoritmo **BuscaProfundidade**(G)

 para cada nó u de G, faça encontrado(u) = False

 para cada nó u de G

 se encontrado(u) == False

BuscaProfundidadeRecursivo(G,u)

Busca em profundidade - recursivo

algoritmo BuscaProfundidadeRecursivo(G,u)

<pre-cond>: G é um grafo (direcionado ou não), e u é um de seus nós.

encontrado(v) é um campo booleano de cada nó v.

encontrado(u) == False.

<pos-cond>: todo nó v alcançável a partir de u por "caminho de não encontrados"

terá encontrado(v) == True.

encontrado(u) = True

para cada vizinho v de u em G

se encontrado(v) == False

BuscaProfundidadeRecursivo(G,v)

algoritmo BuscaProfundidade(G)

para cada nó u de G, faça encontrado(u) = False

para cada nó u de G

se encontrado(u) == False

BuscaProfundidadeRecursivo(G,u)

Caso base: u é o único nó não encontrado alcançável a partir de u.

Busca em profundidade - recursivo

- Medida de tamanho
 - Número de nós alcançáveis a partir de u por caminho de não encontrados
- Subinstância
 - Nó u é marcado como encontrado
 - Nó inicial passa a ser um vizinho v de u
 - Redução de tamanho: pelo menos um nó a menos (nó u)
 - Atende as pré-condições
- Estabelecendo as pós-condições
 - Cada caminho de não encontrados iniciando em u tem como 2o nó um vizinho de u
 - u é marcado como encontrado
 - As pós-condições das chamadas recursivas garantem marcar como encontrado todo alcançável por caminho de não encontrados a partir de um vizinho de u

Busca em profundidade - recursivo

- Caso base:
 - Quando u é “encontrado”, não há caminho de não encontrados iniciando em u
 - Não há tarefa para realizar no caso base

Busca em profundidade - recursivo - complexidade

```
algoritmo BuscaProfundidadeRecursivo(G,u)
    se encontrado(u) == True
        não faz nada
    senão
        encontrado(u) = True
        para cada vizinho v de u em G
            BuscaProfundidadeRecursivo(G,v)                                 $\theta(m)$ 

algoritmo BuscaProfundidade(G)
    para cada nó u de G, faça encontrado(u) = False                         $\theta(n)$ 
    para cada nó u de G
        se encontrado(u) == False                                           $\theta(n)$ 
            BuscaProfundidadeRecursivo(G,u)
```

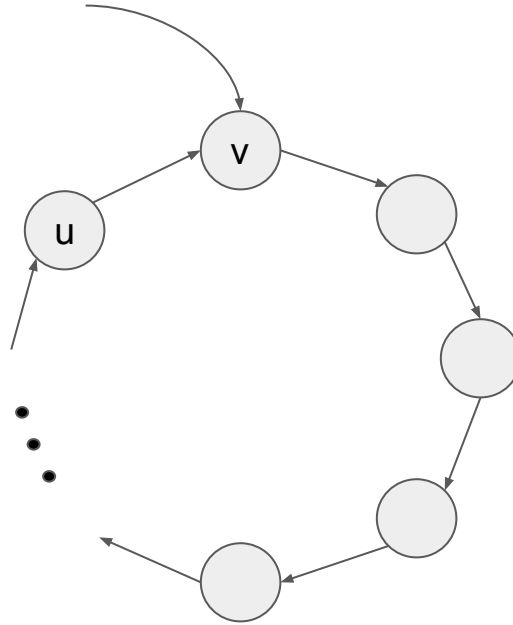
Total: $\theta(n + m)$

Aplicações de busca em profundidade

Aplicações de BP - detecção de ciclos

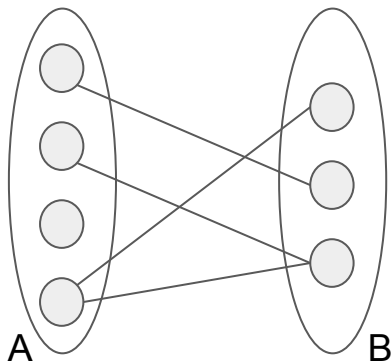
- Um grafo tem ciclo sse tem aresta de retorno
 - ($\Leftarrow$)
 - Se (u,v) é de retorno, então v é ancestral de u na ABP
 - Juntando as arestas de árvore de v até u com (u,v) , formamos um ciclo
 - ($\Rightarrow$)
 - Seja C um ciclo qualquer, e v o 1o nó de C encontrado na busca em profundidade
 - Seja u o nó com aresta para v em C
 - Como todos em C são não encontrados, todos serão descendentes de u na ABP
 - Ao percorrer os vizinhos de u , vamos acessar a aresta (u,v)
 - Como v é ancestral de u , esta aresta será marcada como de retorno

Aplicações de BP - detecção de ciclos



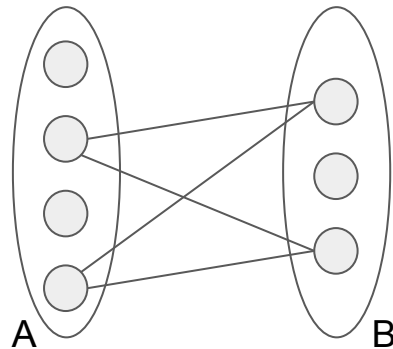
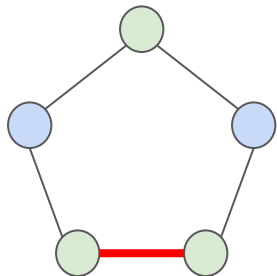
Aplicações de BP - detecção de bipartição

- Grafo não direcionado G é bipartido quando
 - Possível particionar os nós em dois conjuntos A e B tq toda aresta liga nó em A com nó em B
 - Equivalente: podemos colorir os nós com duas cores tq toda aresta conecta cores distintas



Aplicações de BP - detecção de bipartição

- G é bipartido sse não existe ciclo com número ímpar de arestas
 - ($\Rightarrow$) Contrapositiva: se tem ciclo ímpar, então não é bipartido
 - Colorindo alternadamente os nós, vamos sempre obter aresta ligando a mesma cor
 - ($\Leftarrow$) Se não tem ciclo ímpar, então é bipartido
 - Quando nó é encontrado na BP, damos cor oposta ao predecessor (aresta de árvore)
 - Toda aresta de retorno (u,v) conecta nós com cores distintas
 - Pois caso contrário, (u,v) formaria ciclo com número ímpar de arestas
 - Como todas as arestas foram exploradas, o grafo é bipartido

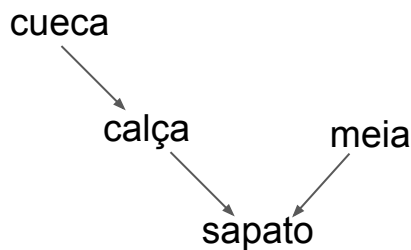


Aplicações de BP - detecção de bipartição

- Detecção de bipartição
 - Busca em profundidade
 - Quando encontrar nó v , coloque v na partição oposta à do pai
 - Quando encontrar aresta de retorno (u,v)
 - Não é bipartido se u e v estão na mesma partição (ciclo ímpar)
 - Se terminar a busca sem este tipo de aresta, então é bipartido
- Se algoritmo retorna “bipartido”, então o grafo é bipartido
 - Como todas as arestas foram exploradas, não existe aresta entre nós de mesma cor
- Se algoritmo retorna “não bipartido”, então o grafo não é bipartido
 - Seja (u,v) uma aresta de retorno identificada entre nós de mesma cor
 - Como u e v estão com mesma cor, temos um número par de arestas de árvore entre u e v
 - Então a aresta (u,v) fecha um ciclo de comprimento ímpar

Aplicações de BP - ordenação topológica

- Ordem total: estabelece ordem para os elementos em um conjunto S tal que
 - Se u e v pertencem a S, ou u está antes de v, ou v está antes de u
 - Transitiva: se u está antes de v, e v está antes de w, então u está antes de w
- Ordem parcial:
 - Estabelece a ordem entre alguns pares, mas outros pares não possuem ordem definida
 - É transitiva
 - Ex.:

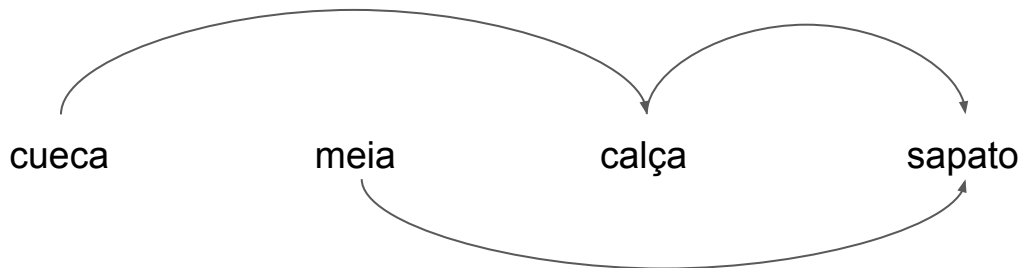


Aplicações de BP - ordenação topológica

- Ordem total: estabelece ordem para os elementos em um conjunto S tal que
 - Se u e v pertencem a S , ou u está antes de v , ou v está antes de u
 - Transitiva: se u está antes de v , e v está antes de w , então u está antes de w
- Ordem parcial:
 - Estabelece a ordem entre alguns pares, mas outros pares não possuem ordem definida
 - É transitiva
 - Pode ser representada por DAGs (grafos direcionados acíclicos)
 - Caminho de u até v indica que u está antes de v
 - Transitividade: se tem caminho de u até v , e de v até w , então tem de u até w
 - Não pode haver ciclos
 - Contradição: u não pode estar ao mesmo tempo antes de depois de v

Aplicações de BP - ordenação topológica

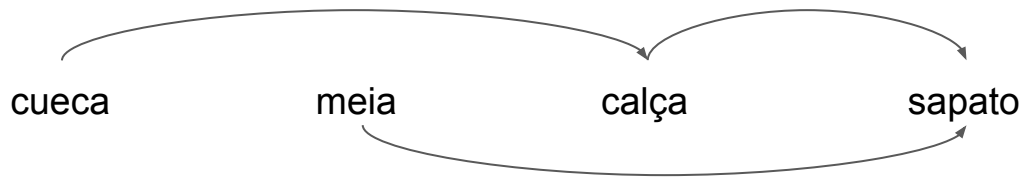
- Problema de ordenação topológica
 - Pré-condições: entrada é um grafo direcionado acíclico G representando uma ordem parcial
 - Pós-condições: saída é uma ordem total consistente com a ordem parcial dada por G
 - Ou seja, para toda aresta (u,v) em G , u aparece antes de v na ordem total.
 - Visualmente: ordem dos nós de modo que toda aresta vai da esquerda para a direita



Aplicações de BP - ordenação topológica

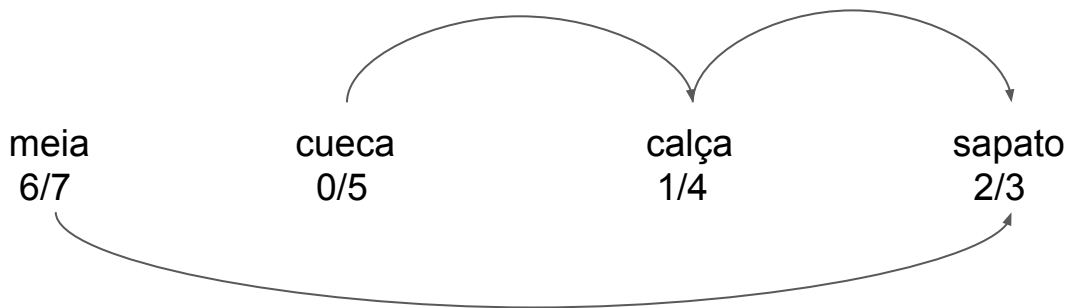
- Algoritmo recursivo

- Sorvedouro: nó sem aresta saindo
 - Todo DAG tem pelo menos um sorvedouro
 - Caso contrário, um caminhamento qualquer iria encontrar nó visitado (ciclo)
 - Pois a aresta de saída permite ir para um novo nó, e número finito de nós
- Iniciando de um nó qualquer, faça um caminhamento qualquer até encontrar um sorvedouro t
- Coloque t no final da ordem total, remova t do grafo, e chame recursivamente
- Tempo: $T(n) = T(n-1) + \theta(n) \in \theta(n^2)$



Aplicações de BP - ordenação topológica

- Algoritmo usando busca em profundidade
 - Execute uma busca em profundidade ($\theta(n+m)$)
 - Retorne os nós em ordem decrescente de f (instante de finalização)
 - Coloque cada nó que finaliza no início de uma lista (não altera complexidade da BP)

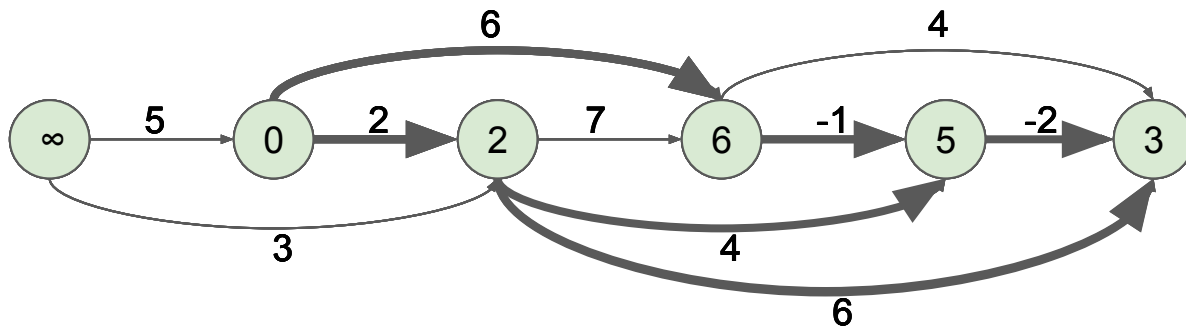


Aplicações de BP - ordenação topológica

- Algoritmo usando busca em profundidade
 - Execute uma busca em profundidade ($\theta(n+m)$)
 - Retorne os nós em ordem decrescente de f (instante de finalização)
 - Coloque cada nó que finaliza no início de uma lista (não altera complexidade da BP)
 - Corretude: Para cada aresta (u,v) de G , v finaliza antes de u na busca em profundidade
 - Se (u,v) é de árvore ou de avanço
 - u é ancestral de v na ABP: $s(u) < s(v) < f(v) < f(u)$
 - Se (u,v) é de retorno
 - Não pode existir, pois G não tem ciclo
 - Se (u,v) é cruzada
 - Quando (u,v) foi acessada, v já havia finalizado, mas u ainda não
 - Então $f(v) < f(u)$

Aplicações de BP - caminho mais curto em DAGs

- Quando entrada é DAG, geralmente facilita executar em ordem topológica
- Algoritmo:
 - Para cada nó u em ordem topológica, relaxe as arestas que saem de u



Aplicações de BP - caminho mais curto em DAGs

- Corretude:
 - Se $P = (s, v_1, v_2, \dots, v_k)$ é um caminho mais curto qualquer de s até v_k
 - Então, a sequência de relaxações $(s, v_1), (v_1, v_2), (v_2, v_3) \dots, (v_{k-1}, v_k)$
 - Mesmo intercalada com outras relaxações
 - Vai produzir o valor correto de d para cada nó em P
 - Como P é mais curto, os i primeiros nós também forma caminho mais curto, para todo i
 - Por indução, se $d(v_i)$ é o tamanho do caminho mais curto de s até v_i ,
 - Então a relaxação de (v_i, v_{i+1}) vai tornar $d(v_{i+1})$ o tam. cam. mais curto de s até v_{i+1}
 - Em todo caminho em P (incluindo os caminhos mais curtos)
 - Os nós aparecem em ordem topológica
 - Então, pela propriedade, e relaxando em ordem topológica
 - Temos que o valor de d é corretamente calculado para todo nó

Aplicações de BP - componentes conexos

- Em grafos não direcionados
 - Relação “alcançáveis” é de equivalência, então particiona os nós em componentes conexos
- Componente conexo: conjunto C de nós tal que
 - Todo par de nós em C são alcançáveis
 - Maximal: inclusão de nó viola a condição anterior
- Iniciando busca em qualquer nó do componente C
 - Vamos alcançar todos os nós em C
 - Nenhum nó fora de C é alcançado
- Ou seja, podemos usar qualquer tipo de busca p/ encontrar comp. conexos

Aplicações de BP - componentes fortemente conexos

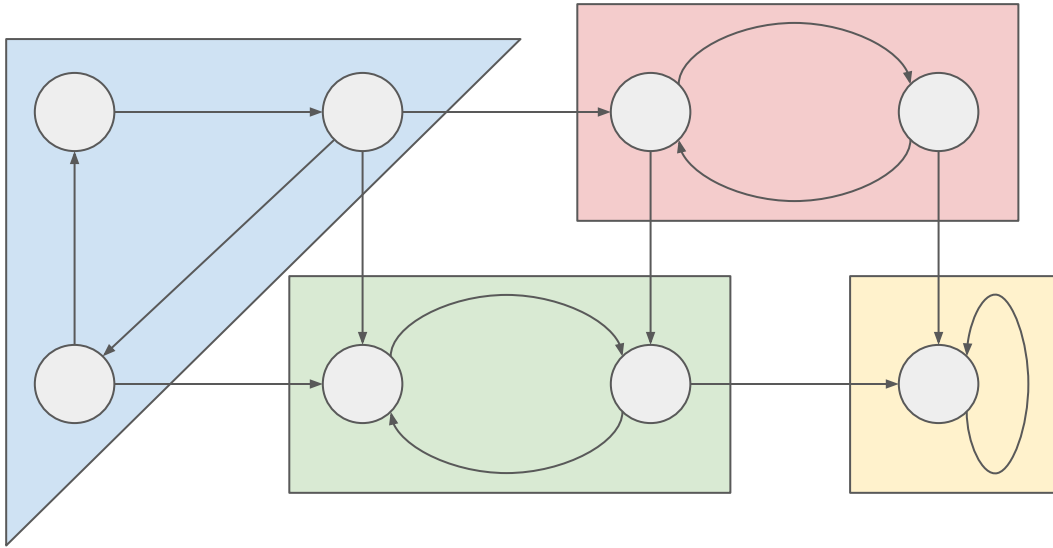
- Em grafos **direcionados**
 - Relação “**mutuamente alcançável**” (u alcança v , e v alcança u) é de equivalência, então particiona os nós em componentes **fortemente** conexos
- Componente **fortemente** conexo: conjunto C de nós tal que
 - Todo par de nós em C são **mutuamente** alcançáveis (u alcança v , e v alcança u)
 - Maximal: inclusão de nó viola a condição anterior

- Algoritmo:

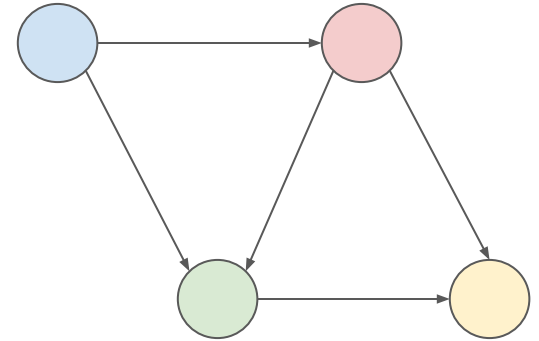
crie um componente para cada vértice
para cada par de vértices u e v
 se u e v são mutuamente alcançáveis
 unir os componentes de u e de v

Veremos algoritmo $\theta(n+m)$
 $\theta(n)$
 $\theta(n^2)$
 $\theta(n+m)$ / iteração (union-find)
 $\theta(1)$ / iteração (union-find)
Total: $\theta(n^2(n+m))$

Aplicações de BP - componentes fortemente conexos



Grafo componente

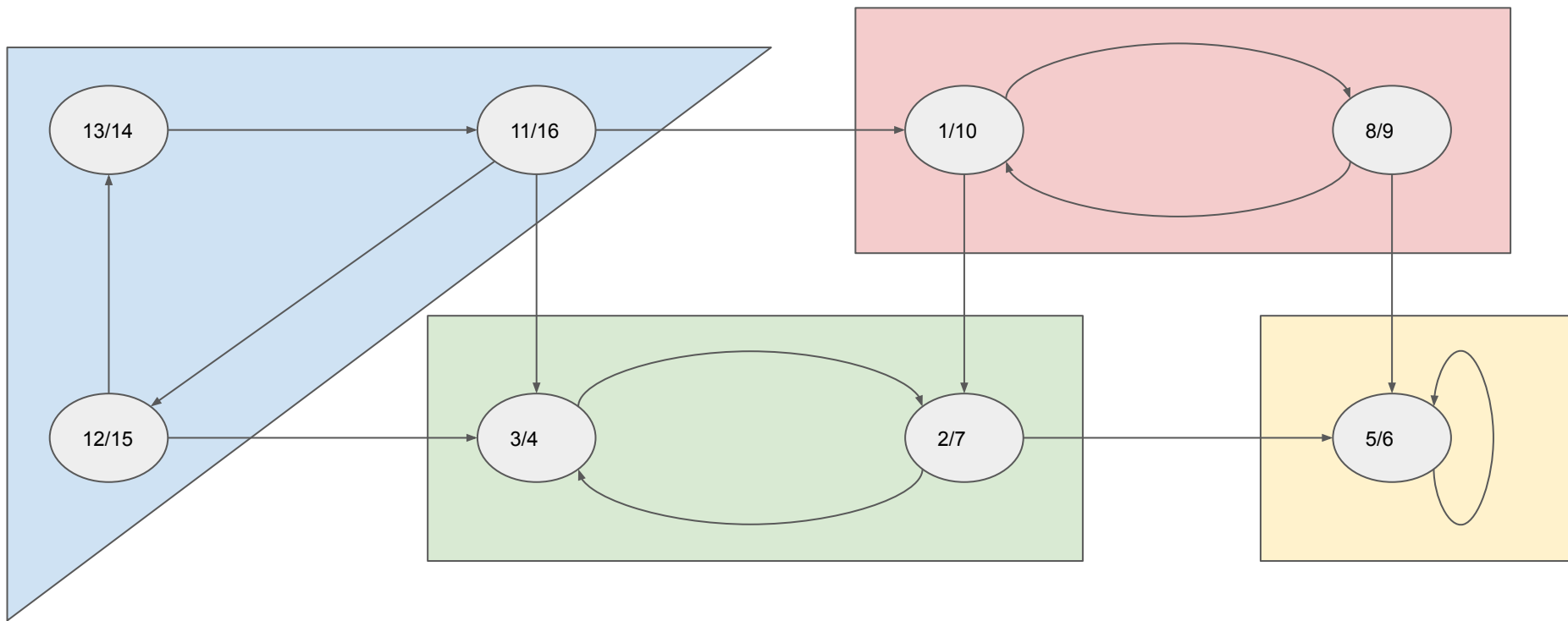


Aplicações de BP - componentes fortemente conexos

- Grafo componente é acíclico
 - Um ciclo iria tornar os componente mutuamente alcançáveis
 - Mas assumimos que os CFC são maximais
- Então admite ordenação topológica e possui componente sorvedouro
- Busca em um componente sorvedouro não sai do componente
- Então podemos remover o componente e continuar recursivamente
- Como encontrar um componente sorvedouro?
 - Mostrarei como encontrar um componente fonte
 - Então invertendo as aresta teremos um componente sorvedouro
 - Note que inversão das arestas conserva os componente (“mutuamente alcançáveis”)

Aplicações de BP - componentes fortemente conexos

- O vértice com maior f em uma BP está em um componente fonte



Aplicações de BP - componentes fortemente conexos

- O vértice com maior f em uma BP está em um componente fonte
 - Contradição: nó u com maior f está em componente C não fonte
 - Então existe componente C' com aresta para C
 - Caso I: primeiro nó encontrado em C ou C' estava em C
 - BP finaliza nós em C antes de iniciar em C' (C não alcança C'), então $f[u]$ não é o maior
 - Caso II: primeiro nó encontrado em C ou C' estava em C'
 - Algum nó em C' é ancestral de u na ABP, então tem f maior



Aplicações de BP - componentes fortemente conexos

COMPONENTES-FORTEMENTE-CONEXOS (G)

execute BP para obter $f[v]$ de todo nó v $\Theta(n+m)$

obtenha o grafo transposto G^T (arestas invertidas) $\Theta(n+m)$

para cada nó u em ordem decrescente de f

se u não foi encontrado

$BP(G^T, u)$, inserindo nós encontrados no próximo CFC $\Theta(n+m)$

- Não precisa ordenar f , basta colocar nó que finaliza no início de uma lista
- $BP(G^T, u)$ não sai do componente, cada aresta e nó é acessado uma vez ao final do laço

